ADR 0127 — An unavailable execution carries a reason, not an error string

- **Status:** Implemented; validation in progress
- **Date:** 2026-09-06
- **Issue:** #666
- **Extends:** ADR 0119

Context

Four local repairs (#658, #662, and two arms in #665) withheld filesystem paths in client responses. The next caller still inherited the same disclosure: `planner::couldnt_run(endpoint, error)` logged an arbitrary `Display` value and also embedded it in an HTTP 500 body unconditionally. The current-main census contains 52 calls in 17 source files.

ADR 0119's conclusion applies one level deeper: **the safety has to live in the value**. Auditing callers alone expires when caller 53 arrives. Moreover, not every caller means that git failed to spawn. Fetch, push and pull also use this helper when git ran but its effects cannot be observed. Redacting all errors to an undifferentiated failure would hide the information needed to avoid retrying an operation whose effects are unknown.

Decision

The signature is `couldnt_run(endpoint, RunFailure, detail)`.

`RunFailure` is a closed enum with 23 payload-free variants and a private mapping to fixed client-safe sentences. A caller cannot supply a formatted path as a reason, even accidentally, because neither `String`, `&str`, nor `Display` implements this vocabulary. There is no raw-text variant, string constructor, or conversion that

asserts arbitrary text is safe. New reasons require editing and reviewing the central enum and its sentence mapping.

The endpoint is log-only. The generic `Display` argument is always detail. The helper delegates composition to `state::withheld_detail`, retaining the established `GIT_VISTA_EXPOSE_PATHS` parser and its policy: an unset or false flag yields the reason alone, and opt-in appends the detail. All existing false spellings produce byte-identical client responses. Every response remains HTTP 500; git refusing a command remains a separate path.

The original detail is logged before trimming for client composition, including leading/trailing whitespace and embedded newlines. Moving the existing log statement before the trim also preserves blank diagnostics; it does not change client formatting. The log is unconditional and byte-identical across flag settings. The flag withholds information from a client, never from the operator.

Each of the 52 call sites selects a reason and retains its entire previous error argument as detail. Most spawn failures share `Spawn`; precondition, path-containment, compare-and-swap, and post-execution observation failures select reasons describing that stage. Post-push/pull uncertainty explicitly says the operation ran and advises checking its state before retrying. The earlier local AddWorktree/RemoveWorktree `withheld_detail` repairs remain valid users of that same mechanism.

Alternatives rejected

- **Wrap only known leaky callers.** This repeats the four local fixes and cannot constrain the next caller.
- **Accept a free-form summary and detail.** A caller can still put an OS/git error in the summary; naming two `&str` arguments does not close the hole.
- **Accept a static string summary.** This excludes ordinary formatting, but still permits a static path or a leaked string

and does not require review of a shared vocabulary.

- **Keep the two-argument helper and always print one generic message.** Safe from disclosure, but loses whether execution happened and what is unknown.
- **Offer an explicitly raw client-text variant.** No audited caller needs unflagged raw text. Adding an escape hatch would create a new default to misuse without satisfying an actual requirement.
- **Recognize path patterns in errors.** Git, gix and OS diagnostics are untrusted detail by origin. A recognizer cannot enumerate every format or every operator-only fact.

This costs one argument at every caller, one central enum, and deliberate review when a new failure stage needs a sentence. Client text is intentionally less specific with the flag off; names and full diagnostics remain in the operator log and in opt-in detail. Opt-in retains the useful reason as well as the original detail rather than preserving the old unstructured body.

Verification

`planner::couldnt_run_suite` invokes the real helper in isolated subprocesses with the real environment flag. No test mutates the parent test process's shared environment. It checks path-bearing `io::Error` text, a path-bearing endpoint, every false spelling, opt-in disclosure, unchanged HTTP status, complete unconditional logs, and useful pre/post-execution explanations.

The same exact-body approach as `worktree_add_suite` and `contract_suite` pins the helper and closed enum. A recursive source census discovers new modules and pins all 52 calls with their reason and detail. There is no hand-maintained list of source files; changes to the boundary, vocabulary or callers become explicit review diffs.

Two `failure-atlas.mutation_check` experiments will run against committed HEAD: one restores unconditional raw-detail disclosure, the other replaces the default safe sentence with

`Error`. Each targets a different behavioral assertion rather than claiming proof from the source pins alone. Run records will be added after validation.

Scope and follow-up

This decision closes `couldnt_run` and every caller of it. It is not a claim that all client error bodies in this server are now path-safe. The audit also found independent raw `Couldn't run git: {e}` formatting in `git_cmd::io_error`, commit-tree paths, and handlers (blame, clone and read). Those do not pass through this helper and need a separate migration/audit; no path recognizer or expanded server-wide policy is introduced silently here.